

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHẦN HIỆU TẠI TP.HCM
BỘ MÔN CÔNG NGHỆ THÔNG TIN



Selenium nâng cao: Waits, POM, kịch bản E2E

Buổi 11 / 12 — Môn: Kiểm thử Phần mềm

Năm học 2025–2026

Nội dung buổi học

- 1 Vị trí trong đề cương — ôn tập buổi 10 và vấn đề mở
- 2 Ba loại wait: implicit, explicit, fluent — tránh `Thread.sleep`
- 3 `ExpectedConditions` hay dùng và stale element trong SPA React
- 4 Page Object Model (POM) — trọng tâm của buổi học
- 5 `PageFactory` và `@FindBy`
- 6 Phân tích một ca kiểm thử E2E: Analyst chặn tài khoản lừa đảo
- 7 Kịch bản phân quyền UI và data-driven E2E với nhiều persona
- 8 Chụp screenshot khi test fail — `TestWatcher`
- 9 Chạy headless và chuẩn bị cho CI (buổi 12)
- 10 Flaky test E2E: nguyên nhân và chiến lược cô lập dữ liệu
- 11 Thực hành — Bài nộp — Tài liệu tham khảo

Chương 6 — Kiểm thử tự động giao diện với Selenium

- **Buổi 10:** Selenium WebDriver, chiến lược locator, thao tác form, test đăng nhập DigiShield
- **Buổi 11 (hôm nay):** waits, Page Object Model, **phân tích một ca kiểm thử hoàn chỉnh với Selenium**, chạy headless

Vị trí trong đề cương (2/2)

Mục tiêu buổi học

- Viết test **ổn định** trên SPA React render bất đồng bộ
- Tổ chức mã test theo POM để **bảo trì được**
- Xây dựng **kịch bản E2E** nghiệp vụ thật trên DigiShield, kết hợp kiểm chéo API

Gắn với BTL (30%)

Sản phẩm buổi này là phần “Selenium E2E” trong báo cáo BTL, demo ở buổi 12.

Ôn tập buổi 10 (1/2)

Đã học

- Kiến trúc WebDriver: test → driver → trình duyệt
- Locator: `By.id`, `By.cssSelector`, `By.xpath`
- Thao tác: `click()`, `sendKeys()`, `clear()`, `getText()`
- Test đăng nhập trang /login của DigiShield

Trang login DigiShield (dev sign-in)

- `#login-role` — 6 vai trò, chọn bằng lớp `Select`: `selectByValue("analyst")`
- `#login-email`, `#login-submit`
- Không có ô mật khẩu: dev profile `permitAll`
- Email tự điền theo vai trò

`npm run dev` không đặt `VITE_COGNITO_*` nên /login hiện dev sign-in form (như buổi 10).

Ôn tập buổi 10 (2/2)

Vấn đề còn treo từ buổi 10

Test đăng nhập **lúc pass lúc fail** dù thao tác đúng. Hôm nay ta mổ xẻ nguyên nhân.

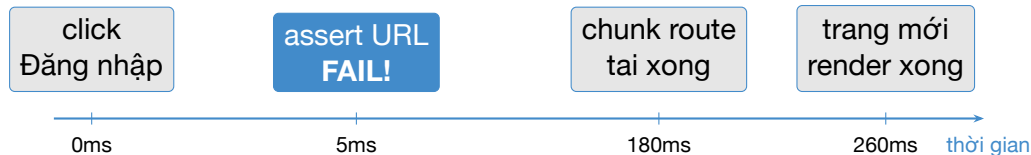
Vấn đề mở: vì sao test login fail?

Nhìn vào code thật của LoginPage.tsx (frontend DigiShield):

```
// src/features/auth/LoginPage.tsx (rut gon)
const onSubmit = (e: FormEvent) => {
  e.preventDefault();
  login({ id: `dev-${role}`, role, email, ... },
        'dev-token');
  navigate(defaultRouteForRole(role));
};
// src/app/router.tsx: moi route deu lazy
const SocInboxPage = lazy(
  () => import('@features/soc/SocInboxPage'));
```

- navigate chạy **ngay**, không có độ trễ nhân tạo
- Nhưng router dùng lazy + Suspense cho **30 route**: trang mới phải **tải chunk JS rồi mới render** ⇒ URL và DOM đổi **sau** cú click

Dòng thời gian: assertion chạy trước khi UI kịp đổi



Bản chất vấn đề

Selenium thao tác **nhANH hơn** ứng dụng phản hồi. Test đúng logic nhưng **sai thời điểm quan sát** $\Rightarrow$ kết quả không ổn định (flaky).

Lời giải của buổi hôm nay

Cơ chế **chờ đợi (waits)**: bảo Selenium “đợi đến khi điều kiện X đúng, tối đa N giây” thay vì quan sát ngay lập tức.

Cách làm sai: Thread.sleep

```
loginButton.click();  
Thread.sleep(3000);    // TUYỆT ĐỐI TRÁNH!  
assertTrue(driver.getCurrentUrl()  
            .contains("/dashboard"));
```

Vì sao TUYỆT ĐỐI tránh Thread.sleep?

- **Chậm:** luôn đợi đủ 3s dù trang xong sau 0.3s — $100 \text{ test} \times 3\text{s} = 5 \text{ phút}$ lãng phí
- **Vẫn flaky:** máy CI chậm, trang mất 3.5s $\Rightarrow$ vẫn fail
- Không diễn đạt **ý định**: đợi cái gì? vì sao 3s?
- **Cộng dồn:** mỗi lần fail lại tăng số giây $\Rightarrow$ suite E2E phình to

Nguyên tắc

Chờ theo **điều kiện** (condition-based), không chờ theo **thời gian cố định** (time-based).

Loại 1: Implicit wait — đặt một lần cho toàn driver (1/2)

```
WebDriver driver = new ChromeDriver();  
// Dat MOT lan, ap dung cho MOI findElement  
driver.manage().timeouts()  
    .implicitlyWait(Duration.ofSeconds(5));  
// Tu day: findElement se poll toi da 5s  
// truoc khi nem NoSuchElementException  
WebElement email =  
    driver.findElement(By.id("login-email"));
```

Loại 1: Implicit wait — đặt một lần cho toàn driver (2/2)

Ưu điểm

- Cấu hình 1 dòng duy nhất
- Giảm fail do element chưa xuất hiện

Nhược điểm

- Chỉ chờ element **tồn tại trong DOM**, không chờ visible/clickable/URL đổi
- Áp dụng mù quáng cho mọi lệnh tìm
- Làm test tiêu cực (element phải biến mất) chậm hẳn

Loại 2: Explicit wait — WebDriverWait (khuyến nghị)

```
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;

// Cho DEN KHI điều kiện dung, tôi đã 5 giây
WebDriverWait wait =
    new WebDriverWait(driver, Duration.ofSeconds(5));

wait.until(ExpectedConditions
    .urlContains("/dashboard"));

WebElement input = wait.until(ExpectedConditions
    .visibilityOfElementLocated(
        By.id("login-email")));
```

Đặc điểm

- Chờ **đúng điều kiện cần tại đúng chỗ cần**
- Poll mỗi 500ms (mặc định); điều kiện đúng là đi tiếp ngay
- Hết timeout $\Rightarrow$ `TimeoutException` với thông điệp rõ ràng

Demo: sửa test login buổi 10 bằng explicit wait (1/2)

Trước (buổi 10) — fail vì điều hướng SPA chưa xong

```
driver.findElement(By.cssSelector(
    "button[type='submit']")).click();
// Assertion chạy ngay lập tức -> URL vẫn /login
assertTrue(driver.getCurrentUrl()
    .contains("/dashboard")); // FAIL
```

Demo: sửa test login buổi 10 bằng explicit wait (2/2)

Sau (buổi 11) — chờ điều kiện URL đổi

```
driver.findElement(By.cssSelector(
    "button[type='submit']")).click();
// Cho toi da 5s cho den khi URL chua /dashboard
new WebDriverWait(driver, Duration.ofSeconds(5))
    .until(ExpectedConditions
        .urlContains("/dashboard")); // PASS on dinh
```

- Thời gian tải chunk + render < 5s timeout $\Rightarrow$ test pass, và chỉ chờ **đúng bằng thời gian thực tế**

Loại 3: Fluent wait — tùy biến polling và ngoại lệ

```
Wait<WebDriver> wait =  
    new FluentWait<>(driver)  
        .withTimeout(Duration.ofSeconds(10))  
        .pollingEvery(Duration.ofMillis(200))  
        .ignoring(NoSuchElementException.class)  
        .withMessage("Watchlist row not visible");  
WebElement row = wait.until(d ->  
    d.findElement(By.xpath(  
        "//div[text()='0909123456']"))));
```

Khi nào dùng fluent wait?

- Cần chu kỳ poll khác 500ms (UI cập nhật rất nhanh/chậm)
- Cần ignoring thêm ngoại lệ (vd `StaleElementReferenceException`)
- Điều kiện lambda tùy biến không có trong `ExpectedConditions`
- `WebDriverWait` chính là `FluentWait` cấu hình sẵn

So sánh các cơ chế chờ

Tiêu chí	Thread.sleep	Implicit	Explicit/Fluent
Chờ theo	thời gian cố định	element có trong DOM	điều kiện bất kỳ
Phạm vi	tại chỗ	toàn driver	tại chỗ, có chủ đích
Tốc độ	luôn chờ đủ	dừng sớm khi thấy	dừng sớm khi đúng
URL/visible/clickable	không	không	có
Độ ổn định	kém	trung bình	cao
Khuyến nghị	cấm	hạn chế	mặc định dùng

Không trộn implicit và explicit wait

Trộn hai loại làm thời gian chờ thực tế **khó dự đoán** (có thể cộng dồn bất thường). Quy ước của lớp: implicit = 0, mọi chỗ cần chờ dùng `WebDriverWait`.

Danh mục ExpectedConditions hay dùng

Điều kiện	Chờ đến khi...
<code>visibilityOfElementLocated(by)</code> <code>elementToBeClickable(by)</code> <code>urlContains(text)</code>	có trong DOM và hiển thị hiển thị và enabled — sắp <code>click()</code> URL chứa chuỗi con — điều hướng SPA
<code>textToBePresentInElementLocated(by, s)</code> <code>presenceOfElementLocated(by)</code> <code>invisibilityOfElementLocated(by)</code> <code>numberOfElementsToBeMoreThan(by, n)</code>	văn bản s xuất hiện trong element có trong DOM (chưa chắc hiển thị) element biến mất — spinner tắt số element > n — bảng đồ dữ liệu

Chọn điều kiện theo ý định kiểm chứng

đọc nội dung → `visibility`; **bấm** → `clickable`; xác nhận **chuyển trang** → `urlContains`.

Ví dụ trên DigiShield: chờ bảng watchlist đổ dữ liệu

Trang /soc/watchlist tải dữ liệu qua TanStack Query (hiển thị “Đang tải watchlist...” trước khi có hàng):

```
WebDriverWait wait =  
    new WebDriverWait(driver, Duration.ofSeconds(5));  
  
// 1. Cho ô tìm kiếm Quick Check hiển thị  
WebElement search = wait.until(  
    ExpectedConditions.visibilityOfElementLocated(  
        By.cssSelector("form input[type='text']")));  
  
// 2. Cho nút "Kiểm tra" bấm được rồi mới click  
wait.until(ExpectedConditions.elementToBeClickable(  
    By.cssSelector("form button[type='submit']")))  
    .click();  
  
// 3. Cho hàng dữ liệu cụ thể xuất hiện trong bảng  
wait.until(ExpectedConditions  
    .visibilityOfElementLocated(By.xpath(  
        "//div[normalize-space()='0909123456']")));
```

StaleElementReferenceException trong SPA React

Hiện tượng

Đã `findElement` thành công, lát sau gọi `getText()`/`click()` thì nổ `StaleElementReferenceException`.

```
WebElement row = driver.findElement(rowLocator);  
// ... React re-render bang (data moi ve) ...  
row.getText();    // StaleElementReferenceException!
```

Nguyên nhân — đặc thù SPA React

- `WebElement` là **con trỏ tới node DOM cụ thể**
- React re-render (state đổi, query refetch) $\Rightarrow$ node cũ bị **gỡ khỏi DOM**, thay bằng node mới trông y hệt
- Con trỏ cũ thành “mồ côi” $\Rightarrow$ stale

Xử lý stale element

Chiến lược 1 — Tìm lại element ngay trước khi dùng

```
// KHÔNG cache WebElement lâu dài;  
// lưu By rồi findElement lại khi cần  
private final By firstRow =  
    By.cssSelector("div[role='button']");  
public String firstRowText() {  
    return driver.findElement(firstRow).getText();  
}
```

Chiến lược 2 — Chờ phiên bản mới của element

```
wait.until(ExpectedConditions.refreshed(  
    ExpectedConditions  
        .visibilityOfElementLocated(firstRow)));
```

- Chiến lược 3: gói việc tìm + thao tác vào **method của Page Object** ⇒ mỗi lần gọi là một lần tìm mới — dẫn ta đến chủ đề trọng tâm: **POM**

Vấn đề: selector rải rác khắp các test

```
@Test void testLogin() {  
    new Select(driver.findElement(By.id("login-role")))  
        .selectByValue("analyst");  
    driver.findElement(By.id("login-email")).clear();  
    driver.findElement(By.id("login-email"))  
        .sendKeys("analyst@dev.digishield.local");  
    driver.findElement(By.id("login-submit")).click();  
    // ... lặp lại ở testWatchlist, testInbox, ...  
}
```

Hệ quả khi UI đổi

Frontend đổi #login-role thành #login-persona ⇒ sửa **hàng chục** test. Selector trùng lặp, test dài, khó đọc, không diễn đạt nghiệp vụ.

Page Object Model — nguyên tắc

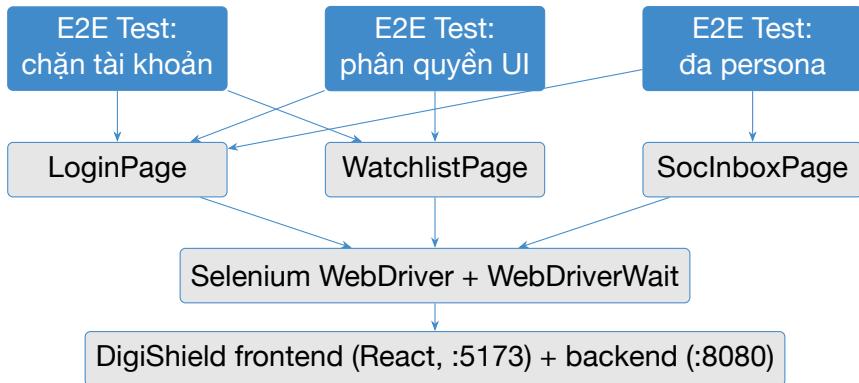
Ba nguyên tắc cốt lõi

- ❶ **Mỗi trang (hoặc thành phần) một class:** LoginPage, WatchlistPage, SocInboxPage... — nơi **duy nhất** biết locator của trang đó
- ❷ **Method mô tả hành vi nghiệp vụ và trả về page object** kế tiếp: `loginAsAnalyst(...)` trả về SocInboxPage $\Rightarrow$ chuỗi thao tác đọc như kịch bản
- ❸ **Test không chứa By:** test chỉ gọi method nghiệp vụ + assert; mọi By, wait, thao tác DOM nằm trong page object

Lợi ích

- UI đổi selector $\Rightarrow$ sửa **một chỗ**
- Test ngắn, đọc được bởi cả người không biết Selenium
- Wait được chuẩn hóa bên trong page object $\Rightarrow$ giảm flaky

Kiến trúc POM cho DigiShield



- Tầng test chỉ nói **nghiệp vụ**; tầng page object nói **DOM**; không tầng nào “nói leo” tầng kia

Cấu trúc thư mục e2e/ trong repo

```
src/test/java/com/digishield/e2e/  
|-- pages/  
|   |-- LoginPage.java  
|   |-- WatchlistPage.java  
|   |-- SocInboxPage.java  
|-- support/  
|   |-- DriverFactory.java          // ChromeOptions  
|   |-- ApiHelper.java             // kiểm tra API  
|   |-- ScreenshotOnFailure.java  
|-- scenarios/  
    |-- AnalystBlocksAccountE2E.java
```

Quy ước đặt tên

- Page object: <Tên trang>Page
- Kịch bản: <Ng nghiệp vụ>E2E — mỗi file một luồng nghiệp vụ
- Đây là cây thư mục **đang có** trong repo; bài tập hôm nay thêm DashboardPage và các kịch bản mới theo đúng quy ước trên

Khảo sát DOM thật trước khi viết POM

Quy tắc vàng: POM mô tả UI thật, không mô tả UI ta tưởng

Mở DevTools trên từng trang, ghi lại selector **thật** trước khi viết code.

Trang	Element	Locator thật
/login	vai trò / email / submit	#login-role (Select), #login-email, #login-submit
/soc/watchlist	ô Quick Check (input tra cứu)	form input[type='text']
/soc/watchlist	giá trị trong bảng	XPath theo text giá trị
/soc/inbox	tab lọc THREAT/SPAM	button[role='tab']

- Chỉ trang login có id ổn định; các trang SOC dùng style inline, **không có id** ⇒ phải dùng CSS cấu trúc / XPath theo text
- Bài học cho dev: nên bổ sung data-testid — nhóm BTL ghi nhận vào báo cáo như một **khuyến nghị testability**

LoginPage (1/2) — locator và khởi tạo

```
public class LoginPage {  
    private static final String URL =  
        "http://localhost:5173/login";  
  
    private final WebDriver driver;  
    private final WebDriverWait wait;  
  
    // Locator that của LoginPage.tsx  
    private final By role = By.id("login-role");  
    private final By email = By.id("login-email");  
    private final By submit = By.id("login-submit");  
  
    public LoginPage(WebDriver driver) {  
        this.driver = driver;  
        this.wait = new WebDriverWait(  
            driver, Duration.ofSeconds(5));  
    }  
}
```

LoginPage (2/2) — hành vi nghiệp vụ

```
public LoginPage open() {
    driver.get(URL);
    wait.until(ExpectedConditions
        .visibilityOfElementLocated(role));
    return this;
}

// roleValue: "analyst"|"learner"|"org_admin"|...
public void loginAs(String roleValue,
    String userEmail) {
    new Select(driver.findElement(role))
        .selectByValue(roleValue);
    WebElement e = driver.findElement(email);
    e.clear(); e.sendKeys(userEmail);
    wait.until(ExpectedConditions
        .elementToBeClickable(submit)).click();
}
```

LoginPage — method trả về page object kế tiếp

```
// Role analyst -> mac dinh ve /soc/inbox
public SocInboxPage loginAsAnalyst(
    String userEmail) {
    loginAs("analyst", userEmail);
    wait.until(ExpectedConditions
        .urlContains("/soc/inbox")); // cho route
    return new SocInboxPage(driver);
}

// Role org_admin -> /dashboard (bai tap hom nay)
public DashboardPage loginAsAdmin(
    String userEmail) {
    loginAs("org_admin", userEmail);
    wait.until(ExpectedConditions
        .urlContains("/dashboard"));
    return new DashboardPage(driver);
}
}
```

- Wait `urlContains` **nằm trong** page object: mọi test dùng login đều được “miễn dịch” với độ trễ tải chunk route

WatchlistPage (1/2) — trang /soc/watchlist

```
public class WatchlistPage {  
    private static final String URL =  
        "http://localhost:5173/soc/watchlist";  
  
    private final WebDriver driver;  
    private final WebDriverWait wait;  
  
    // 0 Quick Check: input text duy nhất trong form  
    // (placeholder: "Nhập số tài khoản, SDT, ...")  
    private final By quickCheckInput =  
        By.cssSelector("form input[type='text']");  
    private final By quickCheckSubmit =  
        By.cssSelector("form button[type='submit']");  
  
    // Hàng dữ liệu: giá trị hiển thị dạng monospace  
    private By rowValue(String value) {  
        return By.xpath(  
            "//div[normalize-space()='\" + value + \"']");  
    }  
}
```

WatchlistPage (2/2) — mở trang, tra cứu, xác nhận

```
public WatchlistPage open() {
    driver.get(URL);
    wait.until(ExpectedConditions.visibilityOfElementLocated(quickCheckInput));
    return this;
}
public WatchlistPage quickCheck(String value) {
    WebElement in = wait.until(ExpectedConditions
        .visibilityOfElementLocated(quickCheckInput));
    in.clear(); in.sendKeys(value);
    wait.until(ExpectedConditions
        .elementToBeClickable(quickCheckSubmit)).click();
    return this;
}
// Cho hàng xuất hiện; hết timeout -> false
public boolean hasEntry(String value) {
    try {
        return wait.until(ExpectedConditions
            .visibilityOfElementLocated(rowValue(value))).isDisplayed();
    } catch (TimeoutException e) { return false; }
}
}
```

SocInboxPage — trang /soc/inbox

```
public class SocInboxPage {
    private final WebDriver driver;
    private final WebDriverWait wait;
    // Tab loc: "THREAT (n)" / "SPAM (n)" / ...
    private By tab(String label) {
        return By.xpath("//button[@role='tab']"
            + "[contains(., '" + label + "')]");
    }
    private final By rows = By.cssSelector("div[role='button']");
    public SocInboxPage filterBy(String label) {
        wait.until(ExpectedConditions.elementToBeClickable(tab(label))).click();
        return this;
    }
    public int reportCount() {
        // Cho bang render lai sau khi loc (chong flaky)
        return wait.until(ExpectedConditions.numberOfElementsToBeMoreThan(rows, 0)).
            size();
    }
    public WatchlistPage gotoWatchlist() {
        return new WatchlistPage(driver).open();
    }
}
```

Test viết lại bằng POM — sạch và diễn đạt nghiệp vụ

Trước: 10+ dòng `findElement` lặp lại

Selector trộn lẫn logic kiểm chứng, không tái sử dụng.

Sau: test đọc như kịch bản nghiệp vụ

```
@Test
void analystSeesThreatReports() {
    SocInboxPage inbox = new LoginPage(driver)
        .open()
        .loginAsAnalyst(
            "analyst@dev.digishield.local");

    inbox.filterBy("THREAT");

    assertTrue(inbox.reportCount() > 0,
        "Phải có ít nhất 1 báo cáo THREAT");
}
```

- Không một By nào lọt vào test — đạt nguyên tắc 3

PageFactory và @FindBy — giới thiệu nhanh

```
public class LoginPageF {
    @FindBy(id = "login-email")
    private WebElement email;

    @FindBy(id = "login-role")
    private WebElement role;

    @FindBy(id = "login-submit")
    private WebElement submit;

    public LoginPageF(WebDriver driver) {
        // Tao proxy: element duoc tim LAI moi lan dung
        PageFactory.initElements(driver, this);
    }

    public void login(String user, String pw) {
        email.clear();    email.sendKeys(user);
        password.clear(); password.sendKeys(pw);
        submit.click();
    }
}
```

@FindBy so với By thủ công

Tiêu chí	@FindBy + PageFactory	By + WebDriverWait
Cú pháp	gọn, khai báo	dài hơn một chút
Lazy lookup	có (proxy tìm lại mỗi lần)	tự viết: tìm trong method
Chống stale	một phần (tìm lại)	chủ động, rõ ràng
Kết hợp wait	khó — proxy không tự chờ điều kiện	tự nhiên: <code>wait.until(...)</code> nhận By
Locator động	không (annotation tĩnh)	có (<code>rowValue(value)</code>)
Debug	khó nhìn thấy locator lúc chạy	dễ log, dễ lần vết

Khuyến nghị của môn học

Dùng By thủ công + explicit wait cho SPA React như DigiShield: cần locator động (giá trị watchlist) và cần wait theo điều kiện — hai thứ PageFactory hỗ trợ kém.

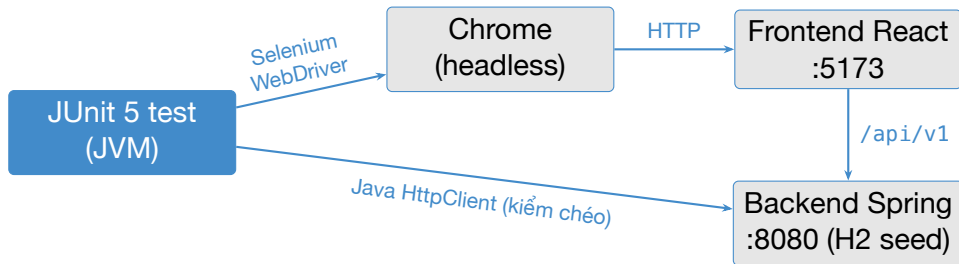
Phân tích một ca kiểm thử E2E (theo đề cương C6)

Kịch bản nghiệp vụ “vàng” của DigiShield

SOC Analyst nhận tin một số điện thoại/tài khoản đang nhận tiền lừa đảo
⇒ đưa vào watchlist để hệ thống **cảnh báo/chặn giao dịch** tới đích đó.

Mã ca	E2E-WL-01 — Analyst chặn tài khoản lừa đảo
Mục tiêu	Luồng chặn hoạt động xuyên suốt UI → API → DB
Tiền điều kiện	Backend dev (H2 seed) + frontend :5173 đang chạy; chưa có giá trị 0909123456 trong watchlist
Dữ liệu	vai trò analyst; analyst@dev.digishield.local; giá trị chặn 0909123456
Các bước	(1) đăng nhập Analyst; (2) thêm entry watchlist; (3) mở /soc/watchlist; (4) tra cứu Quick Check; (5) kiểm chéo API check
Kết quả mong đợi	Hàng 0909123456 hiển thị trong bảng; API trả inWatchlist=true

Thiết kế kỹ thuật: Selenium + kiểm chéo API



Vì sao bước “thêm entry” đi qua API?

Không phải vì UI thiếu — nút “+ Thêm thủ công” của `WatchlistPage.tsx` mở drawer và gọi `POST /blacklist` thật. Test vẫn **chèn dữ liệu qua API** rồi **kiểm chứng qua UI** vì đây là kỹ thuật chuẩn: *setup bằng API — nhanh, ổn định; quan sát bằng UI — đúng trải nghiệm người dùng.*

Hai endpoint watchlist thật của DigiShield

Endpoint	Module	Vai trò
GET/POST /blacklist	reporting	nguồn dữ liệu bảng /soc/watchlist
POST /account-watchlist	interception	kho tài khoản khả nghi cho engine chặn giao dịch
GET .../check?value=	interception	trả {inWatchlist, riskLevel}

Phát hiện khi khảo sát backend

BlacklistType **không có** BANK (chỉ DOMAIN/URL/EMAIL/IP/PHONE/HASH) trong khi UI có nhóm hiển thị “Tài khoản ngân hàng” — form thêm chỉ cho phone/domain/url/email/ip nên nhóm này **luôn rỗng**. Khoảng hờ FE/BE đáng ghi vào báo cáo; demo dùng type="phone".

ApiHelper (1/2) — gọi API với vai ANALYST

```
public class ApiHelper {  
    static final String API =  
        "http://localhost:8080/api/v1";  
    static final String TENANT =  
        "11111111-1111-1111-1111-111111111111";  
    static final HttpClient http =  
        HttpClient.newHttpClient();  
  
    static HttpResponse<String> post(  
        String path, String json) throws Exception {  
        HttpRequest req = HttpRequest.newBuilder()  
            .uri(URI.create(API + path))  
            .header("Content-Type", "application/json")  
            // dev: DevSecurityConfig permitAll  
            .header("X-Tenant-Id", TENANT)  
            .POST(HttpRequest.BodyPublishers  
                .ofString(json))  
            .build();  
        return http.send(req,  
            HttpResponse.BodyHandlers.ofString());  
    }  
}
```

ApiHelper (2/2) — chặn tài khoản và kiểm chéo

```
// Mot hanh dong: ghi vao CA HAI kho (-> 201)
static void blockAccount(String value)
    throws Exception {
    post("/blacklist",
        "{\"type\":\"phone\",\"value\":\"" + value
        + "\",\"source\":\"E2E-BTL\"}");
    post("/account-watchlist", "{\"type\":\"phone\",
        + "\"value\":\"" + value + "\",\"risk_level\"
        + ":\":\"high\", \"source\":\"E2E-BTL\"}");
}
static boolean isInWatchlist(String value)
    throws Exception {
    HttpRequest req = HttpRequest.newBuilder()
        .uri(URI.create(API
            + "/account-watchlist/check?value=" + value))
        .header("X-Tenant-Id", TENANT).GET().build();
    String body = http.send(req, HttpResponse
        .BodyHandlers.ofString()).body();
    return body.contains("\"inWatchlist\":true");
}
}
```

Test E2E-WL-01 (1/2) — khởi tạo driver

```
class AnalystBlocksAccountE2E {  
  
    static WebDriver driver;  
    static final String VALUE = "0909123456";  
  
    @BeforeAll  
    static void startBrowser() {  
        ChromeOptions opts = new ChromeOptions();  
        opts.addArguments("--headless=new",  
                           "--window-size=1920,1080");  
        driver = new ChromeDriver(opts);  
    }  
  
    @AfterAll  
    static void quitBrowser() {  
        if (driver != null) driver.quit();  
    }  
}
```


Test E2E-WL-01 (2/2) — kịch bản đầy đủ dùng POM

```
@Test
void analystBlocksScamAccountEndToEnd()
    throws Exception {
    // (1) Dang nhap role analyst (wait duoc
    //     dong goi trong LoginPage)
    SocInboxPage inbox = new LoginPage(driver)
        .open()
        .loginAsAnalyst(
            "analyst@dev.digishield.local");
    // (2) Chan tai khoan qua API (UI add la stub)
    ApiHelper.blockAccount(VALUE);
    // (3) UI: entry moi hien thi trong bang
    WatchlistPage wl = inbox.gotoWatchlist();
    assertTrue(wl.hasEntry(VALUE),
        "Watchlist phai hien thi " + VALUE);
    // (4) UI: tra cuu nhanh Quick Check
    wl.quickCheck(VALUE);
    // (5) Kiem cheo API tang can thiep
    assertTrue(ApiHelper.isInWatchlist(VALUE),
        "check?value= phai tra inWatchlist=true");
}
```

Đọc lại ca kiểm thử: mỗi bước kiểm chứng điều gì?

Bước	Hành động	Kiểm chứng
(1)	login vai trò analyst	auth flow + điều hướng /soc/inbox (chờ chunk route render)
(2)	POST /blacklist + POST /account-watchlist	API nhận dữ liệu, trả 201
(3)	mở /soc/watchlist	UI đọc GET /blacklist và render đúng hàng mới
(4)	Quick Check trên UI	form tra cứu thao tác được
(5)	GET .../check	engine can thiệp “nhìn thấy” tài khoản: inWatchlist=true

Giá trị của kiểm chéo UI + API

Nếu chỉ assert UI: không biết engine can thiệp có nhận dữ liệu. Nếu chỉ assert API: không biết Analyst có *nhìn thấy* kết quả. E2E tốt kiểm chứng **cả hai mặt** của một hành động nghiệp vụ.

Kịch bản 2: LEARNER bị chặn khỏi trang SOC

Route guard thật trong router.tsx + RequireRole.tsx:

```
// router.tsx: /soc/inbox chỉ cho ANALYST
// <Route path="/soc/inbox"
//   element={guarded(ANALYST, SocInboxPage)} />

// RequireRole.tsx:
// - chưa đăng nhập -> Navigate("/login")
// - sai role -> Navigate("/403")
```

Ca kiểm thử E2E-RBAC-01 (tiêu cực)

- **Arrange:** đăng nhập vai trò **learner** (role learner) → về /learn
- **Act:** gõ thẳng URL /soc/inbox
- **Assert:** bị redirect sang /403, tiêu đề trang (h1) hiển thị “403”
- Song song ở tầng API (buổi 9): gọi endpoint ANALYST bằng role LEARNER → 403 Forbidden — phân quyền phải đúng ở **cả UI lẫn API**

Code kịch bản phân quyền UI

```
@Test
void learnerCannotOpenSocInbox() {
    LoginPage login = new LoginPage(driver).open();
    login.loginAs("learner",
        "learner@dev.digishield.local");
    WebDriverWait wait = new WebDriverWait(
        driver, Duration.ofSeconds(5));
    wait.until(ExpectedConditions
        .urlContains("/learn")); // ve trang learner
    // Co tinh truy cap trang cua ANALYST
    driver.get("http://localhost:5173/soc/inbox");
    // RequireRole redirect ve /403
    wait.until(ExpectedConditions
        .urlContains("/403"));
    assertTrue(driver.findElement(By.tagName("h1"))
        .getText().contains("403"));
}
```

- Test tiêu cực phân quyền là ca **bắt buộc** trong BTL: lỗi phân quyền UI là lỗ hổng bảo mật thường gặp ở SPA

Data-driven E2E: một test — bốn vai trò

Bảng định tuyến thật `defaultRouteForRole` (`roles.ts`) — mỗi vai trò có trang chủ riêng:

```
@ParameterizedTest(name = "{0} -> {2}")
@CsvSource({
    "org_admin,    org_admin@dev.digishield.local,  /dashboard",
    "learner,     learner@dev.digishield.local,    /learn",
    "analyst,     analyst@dev.digishield.local,    /soc/inbox",
    "super_admin, super_admin@dev.digishield.local, /super/tenants"
})
void loginLandsOnRoleHome(String role,
    String email, String route) {
    new LoginPage(driver).open()
        .loginAs(role, email);
    new WebDriverWait(driver, Duration.ofSeconds(5))
        .until(ExpectedConditions
            .urlContains(route));
}
```

- Kỹ thuật `@ParameterizedTest` của buổi 5 tái xuất ở tầng E2E: 4 ca kiểm thử, một thân test duy nhất

Chụp màn hình bằng TakesScreenshot

```
// Mọi ChromeDriver đều là TakesScreenshot
File shot = ((TakesScreenshot) driver)
    .getScreenshotAs(OutputType.FILE);

Files.createDirectories(
    Path.of("build", "screenshots"));
Files.copy(shot.toPath(),
    Path.of("build", "screenshots",
        "watchlist-fail.png"),
    StandardCopyOption.REPLACE_EXISTING);
```

Vì sao screenshot quan trọng với E2E?

- Test E2E fail trên CI headless: **không ai nhìn thấy** trình duyệt lúc đó
- Ảnh chụp tại thời điểm fail = bằng chứng số 1 để chẩn đoán (trang trắng? toast lỗi? còn ở /login?)
- Đính kèm vào báo cáo BTL làm minh chứng thực thi

Tự động chụp khi fail: JUnit 5 TestWatcher

```
public class ScreenshotOnFailure implements TestWatcher {
    @Override
    public void testFailed(ExtensionContext ctx, Throwable cause) {
        WebDriver driver = DriverFactory.current();
        if (driver == null) return;
        try {
            File shot = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
            Path dir = Path.of("build", "screenshots");
            Files.createDirectories(dir);
            String name = ctx.getDisplayName()
                .replaceAll("[^a-zA-Z0-9_-]", "_");
            Files.copy(shot.toPath(), dir.resolve(name + ".png"),
                StandardCopyOption.REPLACE_EXISTING);
        } catch (IOException e) {
            System.err.println("No screenshot: " + e);
        }
    }
}

// Trong test class:
@ExtendWith(ScreenshotOnFailure.class)
class AnalystBlocksAccountE2E { ... }
```

Báo cáo kết quả suite E2E

Nguồn báo cáo có sẵn từ Gradle

- `./gradlew test` $\Rightarrow$ HTML report:
`build/reports/tests/test/index.html`
- XML JUnit (`build/test-results/`) — đầu vào cho CI (buổi 12)
hiển thị pass/fail từng ca
- Thư mục `build/screenshots/` — ảnh các ca fail

Nội dung phần E2E trong báo cáo BTL

- 1 Bảng đặc tả kịch bản (mã ca, bước, dữ liệu, kết quả mong đợi —
khuôn mẫu buổi 9)
- 2 Sơ đồ POM: trang nào, class nào, method nào
- 3 Kết quả chạy: ảnh HTML report + screenshot minh chứng
- 4 Lỗi/khoảng hở phát hiện được (vd nhóm “Tài khoản ngân hàng” luôn
rỗng vì `BlacklistType` thiếu `BANK`; thiếu `data-testid`)

Chạy headless: `--headless=new`

```
public class DriverFactory {  
    public static WebDriver createHeadless() {  
        ChromeOptions opts = new ChromeOptions();  
        opts.addArguments(  
            "--headless=new",           // headless che do moi  
            "--window-size=1920,1080", // co dinh viewport!  
            "--disable-gpu",  
            "--no-sandbox");           // can cho CI container  
        return new ChromeDriver(opts);  
    }  
}
```

Ghi chú

- `--headless=new`: chế độ headless hợp nhất của Chrome 109+, gần Chrome thật hơn chế độ cũ
- `--window-size` **bắt buộc**: viewport headless mặc định nhỏ $\Rightarrow$ layout đổi, element bị ẩn $\Rightarrow$ test fail khó hiểu
- Debug: bỏ 2 dòng đầu là thành chế độ có giao diện

Headless và CI: tốc độ, ổn định (chuẩn bị buổi 12)

Vì sao headless cho CI?

- Máy CI **không có màn hình**
- Nhanh hơn 20–40% (không vẽ UI)
- Chạy song song nhiều instance
- Ảnh chụp vẫn hoạt động bình thường

Trình tự pipeline (buổi 12)

- 1 Khởi động backend dev (H2 seed)
- 2 Build + serve frontend
- 3 Chạy suite E2E headless
- 4 Thu HTML report + screenshots

Quy tắc chọn số lượng ca E2E

E2E **đắt** (chậm, dễ flaky, khó bảo trì) — theo Testing Pyramid chỉ giữ **5–10%** tổng số test. Chỉ tự động hóa **luồng nghiệp vụ vàng**: đăng nhập, chặn tài khoản, làm quiz, báo cáo phishing. Biến thể và nhánh lỗi ⇒ đẩy xuống unit/integration/API test (buổi 5–9).

Flaky test: nguyên nhân thường gặp

Nguyên nhân	Biểu hiện trên DigiShield	Khắc phục
Wait sai / thiếu	login chờ chunk route; bảng đồ dữ liệu chậm	explicit wait đúng điều kiện
Dữ liệu phụ thuộc	test A thêm 0909123456, test B đếm số hàng $\Rightarrow$ lệch	giá trị ngẫu nhiên/tiền tố riêng, reset seed
Animation CSS	các trang có fadeUp .3s — element “đang bay” khi click	chờ clickable; tắt animation khi test
Phụ thuộc thứ tự test	pass khi chạy lẻ, fail khi chạy suite	mỗi test tự chuẩn bị dữ liệu
Môi trường CI chậm	pass local, fail CI	timeout hợp lý (5–10s), headless cố định viewport

Flaky = mất niềm tin: một suite E2E flaky sẽ bị cả đội **bỏ qua kết quả** — tệ hơn là không có test. Ổn định quan trọng hơn số lượng.

Chiến lược cô lập dữ liệu cho E2E

Tận dụng H2 in-memory của profile dev

```
./gradlew :boot:app:bootRun \  
  --args='--spring.profiles.active=dev'
```

- H2 **in-memory**: khởi động backend là có ngay bộ dữ liệu demo seed sạch — restart = reset toàn bộ
- Trước mỗi **phiên** chạy suite E2E: khởi động lại backend $\Rightarrow$ trạng thái đầu luôn như nhau

Trong từng test

```
// Giá trị riêng cho mỗi lần chạy -> không dùng  
// do các lần chạy trước / test khác  
String value = "0909" + System.currentTimeMillis()  
               % 1_000_000;
```

- Assert theo **giá trị cụ thể vừa thêm**, tránh assert theo **tổng số hàng** (phụ thuộc seed)

Thực hành 1: hoàn thiện POM cho 3 trang

Chuẩn bị môi trường (như buổi 10)

```
# Làm việc trên main – không cần checkout revision cũ
# Terminal 1 – backend dev (H2 seed)
./gradlew :boot:app:bootRun \
    --args='--spring.profiles.active=dev'
# Terminal 2 – frontend
cd frontend && npm run dev      # :5173
```

Yêu cầu (45 phút)

- 1 Tạo package pages/: LoginPage, WatchlistPage, SocInboxPage theo slide; thêm DashboardPage (urlContains("/dashboard"))
- 2 Mọi wait dùng WebDriverWait – cấm Thread.sleep (GV sẽ grep!)
- 3 Viết test analystSeesThreatReports (slide POM), chạy pass ở chế độ có giao diện
- 4 Đổi thử một locator thành sai → quan sát TimeoutException

Thực hành 2: watchlist E2E, chạy headless

Yêu cầu (45 phút)

- 1 Viết ApiHelper (POST /blacklist; POST + GET /account-watchlist/check)
- 2 Cài đặt ca **E2E-WL-01** “Analyst chặn tài khoản” theo 5 bước đã phân tích, dùng POM
- 3 Cài đặt ca **E2E-RBAC-01**: Learner truy cập /soc/inbox → redirect /403
- 4 Chuyển cả hai ca sang **headless** và chạy lại — so sánh thời gian
- 5 Gắn `@ExtendWith(ScreenshotOnFailure.class)`; cố tình làm 1 assert sai để kiểm tra ảnh được chụp

Câu hỏi thảo luận

Giá trị test dùng `System.currentTimeMillis()` — vì sao? Điều gì xảy ra nếu 2 sinh viên chạy cùng lúc trên **cùng một** backend?

Bài nộp (về nhà — gắn với BTL, hạn: trước buổi 12)

Yêu cầu bắt buộc

- 1 $\geq$ **2 kịch bản E2E theo POM** cho module BTL của nhóm: ít nhất 1 **luồng nghiệp vụ vàng** + 1 **tiêu cực** (phân quyền / dữ liệu sai)
- 2 Toàn bộ chạy **headless**; không có `Thread.sleep` trong code
- 3 **Screenshot tự động khi fail** + ảnh HTML report
- 4 Bảng đặc tả từng ca (khuôn mẫu buổi 9): mã ca, bước, dữ liệu, kết quả mong đợi/thực tế
- 5 Mục “Phát hiện”: lỗi/khuyến nghị testability tìm được (thiếu `data-testid`, nút stub, khoảng hở FE/BE)

Nộp

Push vào repo nhóm, thư mục `e2e/`; báo cáo PDF cập nhật chương Selenium. Demo trực tiếp trong buổi bảo vệ BTL (buổi 12).

Tóm tắt buổi 11 và hẹn buổi 12

Đã học hôm nay

- 3 loại wait; `WebDriverWait` + `ExpectedConditions` là mặc định; cấm `Thread.sleep`
- Stale element trong SPA React và cách xử lý
- POM: mỗi trang một class, method trả page object, test không chứa By
- Kịch bản E2E kết hợp Selenium + kiểm chéo API
- Screenshot khi fail, headless, chống flaky

Buổi 12 — buổi cuối

Chương 7: Kiểm thử trong thực tiễn

- Smoke / regression / security / performance
- Kiểm thử hệ thống AI (module ai)
- CI/CD: chạy toàn bộ test tự động
- Tổng kết + **bảo vệ BTL**

Tài liệu tham khảo

- [1] *Slide bài giảng điện tử môn Kiểm thử Phần mềm.*
- [2] Paul Ammann & Jeff Offutt, *Introduction to Software Testing*, Cambridge University Press, 2008.
- [3] Glenford J. Myers, *The Art of Software Testing*, 2nd ed., Wiley, 2004.
- [4] ISTQB Foundation Level Syllabus, version 4.0, 2023. <https://istqb.org/>
- [5] Selenium Project, *Selenium WebDriver Documentation: Waits*, <https://www.selenium.dev/documentation/webdriver/waits/>.
- [6] Selenium Project, *Test Practices: Page Object Models*, https://www.selenium.dev/documentation/test_practices/.
- [7] JUnit Team, *JUnit 5 User Guide* (Extensions, TestWatcher, ParameterizedTest), <https://junit.org/junit5/docs/current/user-guide/>.